

BENUTZERHANDBUCH & CODE-REZEPT

Flashback

Wie aus Grund und Boden ein ganzes Haus wurde

Die Lernkartei-App Schritt fuer Schritt erklart

Vue 3 · Vite · SCSS · Pinia · Supabase · Anthropic API

Projekt: flashcard-app (Flashback)

Erstellt fuer: Amyra

Ein vereinfachtes, projektbezogenes Nachschlagewerk

So liest du dieses Handbuch

Stell dir dein Projekt wie ein Haus vor. Man faengt nicht beim Dach an, sondern beim Grundstueck und dem Fundament. Erst danach kommen Waende, Leitungen, Raeume und ganz zum Schluss die Einrichtung. Genau in dieser Reihenfolge gehen wir auch deine App durch – vom **Fundament** (Vue + Vite) ueber die **Versorgungsleitungen** (Supabase, API) bis zu den **Raeumen** (deinen Views) und der **Einrichtung** (dem SCSS-Design).

Die Bau-Metapher in diesem Handbuch

Fundament = Vue 3 + Vite + die Datei main.js. Ohne das steht nichts.

Versorgung = Supabase (Datenbank) und der Express-Proxy fuer die KI. Wie Strom und Wasser.

Leitungen = die Pinia-Stores. Sie transportieren Daten durch das ganze Haus.

Innenausbau = das SCSS-Design-System (Farben, Abstaende, Glas-Optik).

Moebel = wiederverwendbare Komponenten (z. B. eine Flashcard).

Raeume = die Views, also die einzelnen Seiten, die du im Browser aufrufst.

Legende der Kaesten

Blaue Kaesten erklaren ein Konzept oder fassen etwas zusammen.

Gruene Kaesten sind Tipps und gute Gewohnheiten.

Rote Kaesten sind Warnungen – typische Fehlerquellen.

Dunkle Bloecke zeigen echten Code aus deinen Dateien. Darueber steht immer der Dateiname.

Hinweis zu den Emojis im Code

Dein Code enthaelt viele Emojis (z. B. in Buttons). Eine PDF-Schrift kann farbige Emojis nicht darstellen. Deshalb erscheinen sie hier als Kuerzel in eckigen Klammern, z. B. [Lupe] statt der Lupe oder [Muelleimer] statt des Papierkorbs. In deinem echten Code stehen weiterhin die Emojis.

Inhalt

- 1** Die Vogelperspektive – was ist Flashback?
- 2** Das Grundstueck & Fundament – Vue, Vite, main.js
- 3** Die Versorgung – Supabase, .env und der API-Proxy
- 4** Das Leitungsnetz – Pinia-Stores & Datenfluss
- 5** Der Innenausbau – das SCSS-Design-System
- 6** Die Moebel – wiederverwendbare Komponenten
- 7** Die Raeume – deine Views im Steckbrief
- 8** Sechs Views im Detail – Schritt fuer Schritt
- 9** Der smarte Helfer – BlueBooster & KI-Validierung
- 10** Komfort & Technik – PWA, Toasts, VueUse, Routing
- 11** Glossar – englische Begriffe einfach erklart
- 12** Anhang – die komplette Ordnerstruktur

KAPITEL 1

Die Vogelperspektive - was ist Flashback?

Flashback ist deine persoenliche Lernkartei-App. Man kann damit Lernkarten („Flashcards“) anlegen, Notizen schreiben, taeglich eine Frage beantworten und sich von einem KI-Assistenten namens **BlueBooster** beim Lernen helfen lassen. Bevor wir in den Maschinenraum steigen, schauen wir von oben auf das ganze Haus.



Abbildung 1: Die Bau-Metapher. Jede Etage entspricht einem Teil deines Projekts – von unten (Versorgung) nach oben (KI-Assistent).

Womit wurde das Haus gebaut? (Der Tech-Stack)

Ein **Tech-Stack** (englisch fuer „Technologie-Stapel“) ist die Liste aller Werkzeuge und Bausteine, aus denen die App besteht. Die folgende Tabelle stammt direkt aus deiner `package.json` und deinem `README`.

Baustein	Rolle im Haus	Wofuer im Projekt
Vue 3	Tragwerk	Das Frontend-Framework. Baut die sichtbare Oberflaeche aus Komponenten.
Vite	Bau-Maschine	Build-Tool und Entwicklungs-Server. Startet die App blitzschnell.
SCSS	Innenausbau	Erweitertes CSS. Liefert das ganze Design-System (Farben, Glas-Optik).
Tailwind	Werkzeugkasten	Hilfs-CSS-Klassen („Utility-First“) als Ergaenzung.
Pinia	Leitungsnetz	State-Management: speichert Daten zentral fuer die ganze App.
Supabase	Wasser/Strom	Die Datenbank (PostgreSQL) in der Cloud. Speichert Karten, Notizen, Fragen.
Vue Router	Tueren/Flure	Wechselt zwischen den Seiten, ohne neu zu laden.
TipTap	Schreibtisch	Rich-Text-Editor fuer Notizen und Antworten.
VueUse	Schweizer Messer	Fertige Helfer wie Dark-Mode und Debounce.
Express	Pfoertner	Kleiner Server, der den geheimen API-Schluessel schuetzt.
Anthropic API	Hausmeister-KI	Erzeugt Fragen, prueft Antworten, treibt BlueBooster an.
vite-plugin-pwa	Schluessel	Macht die App installierbar (Progressive Web App).

Was bedeutet „Frontend“ und „Backend“?

Frontend („vordere Seite“) ist alles, was du im Browser siehst und anklickst – in deinem Fall alles, was Vue baut.

Backend („hintere Seite“) ist alles im Hintergrund: die Datenbank (Supabase) und dein kleiner Express-Server, der mit der KI spricht.

KAPITEL 2

Das Grundstueck & Fundament

Jedes Haus braucht zuerst ein Fundament. Bei einer Vue-App sind das vier Dinge, die zusammen dafür sorgen, dass überhaupt etwas im Browser erscheint: die `index.html`, die `main.js`, die Wurzel-Komponente `App.vue` und das Bau-Werkzeug **Vite**.

2.1 Der Einstiegspunkt: `index.html`

Diese Datei ist das leere Grundstueck. Sie enthaelt fast nichts – nur ein einziges leeres `<div>` mit der id `app`. Genau in dieses `div` „haengt“ Vue spaeter das ganze Haus hinein.

Datei: `index.html`

```
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.js"></script>
</body>
```

Das `<script>`-Tag laedt deine `main.js`. Das ist der Startknopf der gesamten Anwendung.

2.2 Der Startknopf: `main.js`

In der `main.js` wird die App zusammengesteckt: Vue wird gestartet und alle **Plugins** (Zusatzbausteine) werden „angemeldet“. Ein Plugin ist ein Werkzeug, das du der App einmal bekannt machst und dann ueberall benutzen kannst.

Datei: `src/main.js`

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import piniaPluginPersistedstate from 'pinia-plugin-persistedstate'
import router from './router/index.js'
import Toast from 'vue-toastification'
import './styles/main.scss'
import App from './App.vue'

// Pinia mit Persistenz-Plugin einrichten
const pinia = createPinia()
pinia.use(piniaPluginPersistedstate)

// App erstellen und alle Plugins registrieren
const app = createApp(App)
app.use(pinia)
app.use(router)
app.use(Toast, { position: 'top-right', timeout: 3000 })
app.mount('#app')
```

Zeile fuer Zeile, in einfachen Worten:

- `createApp(App)` nimmt deine Wurzel-Komponente `App.vue` und macht daraus eine echte App.
- `app.use(pinia)` schaltet das Leitungsnetz (Datenspeicher) ein. Das **Persistenz-Plugin** sorgt dafür, dass die Daten einen Seiten-Neustart ueberleben.
- `app.use(router)` schaltet die Tueren ein, damit man zwischen Seiten wechseln kann.
- `app.use(Toast, ...)` aktiviert kleine Hinweis-Meldungen („Toasts“), die oben rechts erscheinen.

- `import './styles/main.scss'` zieht das gesamte Design-System herein.
- `app.mount('#app')` haengt das fertige Haus in das leere div aus der index.html.

Was ist ein „import“?

`import` (englisch fuer „einfuehren“) holt Code aus einer anderen Datei oder einem Paket herein, damit du ihn hier benutzen kannst. So bleibt jede Datei klein und uebersichtlich.

2.3 Die Wurzel-Komponente: App.vue

App.vue ist der Rohbau, der auf jeder Seite gleich bleibt: oben die Navigationsleiste, in der Mitte der wechselnde Inhalt, unten der Fuss und der schwebende BlueBooster. Der Platzhalter `<RouterView />` ist das Loch in der Wand, durch das der Router die jeweils passende Seite einsetzt.

Datei: `src/App.vue` (Template)

```
<template>
  <div class="app-wrapper">
    <AppNavbar />
    <main class="app-main">
      <RouterView />  <!-- hier erscheint die aktuelle Seite -->
    </main>
    <AppFooter />
    <BlueBooster />
  </div>
</template>
```

2.4 Das Bau-Werkzeug: Vite

Vite startet waehrend der Entwicklung einen lokalen Server und baut am Ende die fertige App. In deiner `vite.config.js` sind drei wichtige Dinge eingestellt: das Vue-Plugin, das PWA-Plugin und ein **Proxy**.

Datei: `vite.config.js` (Auszug)

```
export default defineConfig({
  plugins: [ vue(), tailwindcss(), VitePWA({ /* ... */ }) ],

  // Proxy: leitet /api-Anfragen an lokalen Express-Server weiter
  server: {
    proxy: { '/api': 'http://localhost:3001' }
  }
})
```

Was ist ein „Proxy“?

Ein **Proxy** („Stellvertreter“) ist ein Vermittler. Wenn dein Frontend etwas an `/api` schickt, faengt Vite das ab und leitet es heimlich an deinen Express-Server auf `localhost:3001` weiter. Mehr dazu in Kapitel 3.

KAPITEL 3

Die Versorgung - Supabase, .env und der API-Proxy

Ein Haus ohne Strom und Wasser ist nur eine leere Huelle. Bei Flashback kommen „Strom und Wasser“ aus zwei Quellen: aus **Supabase** (deine Datenbank) und aus der **Anthropic API** (die KI). Beide brauchen geheime Schluessel – und die muessen gut versteckt sein.

3.1 Supabase – die Datenbank in der Cloud

Supabase ist ein Online-Dienst, der dir eine fertige PostgreSQL-Datenbank gibt. Deine App verbindet sich mit nur einer kleinen Datei damit. Diese Datei ist ein **Composable** – also eine wiederverwendbare Helfer-Funktion.

Datei: `src/composables/useSupabase.js`

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY

export const supabase = createClient(supabaseUrl, supabaseAnonKey)
```

Hier passiert genau eines: Es wird ein **Client** (eine Verbindung) zu Supabase erstellt und als `supabase` exportiert. Jeder Store (Kapitel 4) importiert dieses eine Objekt und kann damit Daten lesen und schreiben.

Was bedeutet `import.meta.env`?

`import.meta.env.VITE_SUPABASE_URL` liest einen Wert aus deiner geheimen `.env`-Datei. So steht die Adresse deiner Datenbank nicht direkt im Code.

Wichtig: Nur Variablen, die mit `VITE_` beginnen, darf das Frontend lesen. Das ist eine Sicherheitsregel von Vite.

3.2 Die Geheimnisse: `.env`

Die `.env`-Datei ist der Tresor des Hauses. Hier liegen die Schluessel. Deine `.env.example` zeigt, welche Schluessel gebraucht werden – aber ohne echte Werte:

Datei: `.env.example`

```
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
ANTHROPIC_API_KEY=
```

Niemals den Tresor offen liegen lassen

Die echte `.env` darf **niemals** in Git landen. Deshalb steht sie in der `.gitignore`. Besonders der `ANTHROPIC_API_KEY` ist heikel – mit ihm koennte jemand auf deine Kosten die KI nutzen.

3.3 Der Pfoertner: der Express-Proxy fuer die KI

Der KI-Schlüssel darf nie im Frontend stehen, denn jeder koennte ihn im Browser auslesen. Deshalb gibt es einen kleinen eigenen Server: `server.cjs`. Er nimmt die Anfrage deiner App entgegen, haengt den geheimen Schlüssel an und spricht erst dann mit Anthropic.

Datei: `server.cjs` (Kern)

```
app.post('/api/chat', async (req, res) => {
  const { messages, system } = req.body

  const response = await fetch('https://api.anthropic.com/v1/messages', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'x-api-key': process.env.ANTHROPIC_API_KEY, // Schlüssel bleibt hier
      'anthropic-version': '2023-06-01'
    },
    body: JSON.stringify({
      model: 'claude-sonnet-4-6',
      max_tokens: 1024,
      ...(system ? { system } : {}),
      messages
    })
  })

  const data = await response.json()
  return res.status(200).json(data)
})

app.listen(3001, () => console.log('API-Server laeuft auf http://localhost:3001'))
```

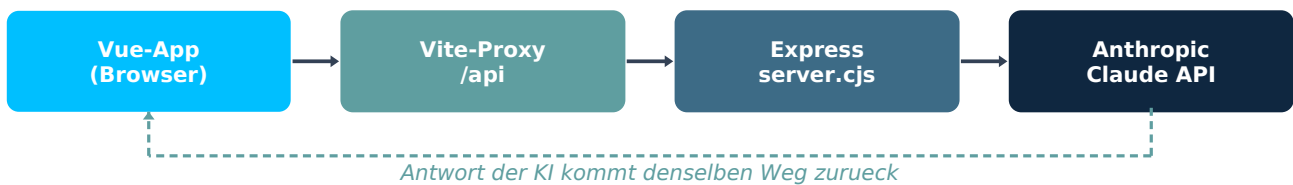


Abbildung 2: Der Weg einer KI-Anfrage. Der geheime Schlüssel wird erst im Express-Server angehaengt – das Frontend sieht ihn nie.

Damit Frontend *und* Server gleichzeitig laufen, gibt es in der `package.json` das Kommando `npm run start`. Es startet beide auf einmal (mit dem Werkzeug „concurrently“).

Typische Fehlerquelle

Startest du nur `npm run dev`, laeuft der Express-Server nicht. Dann funktionieren BlueBooster und der Fragenkatalog nicht, weil `/api/chat` ins Leere geht. Nutze immer `npm run start`.

Gut zu wissen: zwei Server-Dateien

Es gibt `server.cjs` (fuer lokal, mit Express) und `api/chat.js` (eine „Serverless Function“ fuer das spaetere Veroeffentlichen, z. B. bei Vercel). Beide tun dasselbe: den Schlüssel verstecken.

KAPITEL 4

Das Leitungsnetz - Pinia-Stores & Datenfluss

Jetzt kommen die wichtigsten Leitungen des Hauses: die **Stores**. Ein Store ist ein zentraler Daten-Schrank, auf den jede Seite zugreifen kann. Ohne Stores müsste jede Komponente ihre Daten selbst von Supabase holen – mit Stores holt sie eine Stelle ab, und alle teilen sich dieselben Daten.

Warum Pinia? (State-Management einfach erklärt)

State („Zustand“) sind die aktuellen Daten der App: z. B. deine Liste an Flashcards.

State-Management heisst: diese Daten an einer zentralen Stelle ordentlich verwalten.

Pinia ist das Werkzeug dafür in Vue. Ein Pinia-Store hat drei Teile: **State** (die Daten), **Actions** (Funktionen, die die Daten ändern) und optional **Getters** (berechnete Werte aus den Daten).

4.1 Anatomie eines Stores am Beispiel flashCardStore.js

Dein Flashcard-Store ist das beste Beispiel, weil hier das volle Muster sichtbar ist: laden, hinzufügen, löschen und als gelernt markieren. Schauen wir uns zuerst den **State** an.

Datei: `src/stores/flashCardStore.js` (**State**)

```
export const useFlashcardStore = defineStore('flashcards', () => {

  // --- State ---
  const flashcards = ref([])    // die Liste aller Karten
  const isLoading = ref(false) // lädt gerade etwas?
  const error      = ref(null)  // gab es einen Fehler?
```

`ref([])` macht aus einer normalen Variable eine **reaktive** Variable. „Reaktiv“ heisst: Sobald sich der Wert ändert, aktualisiert Vue automatisch alle Stellen auf dem Bildschirm, die diesen Wert anzeigen. Du musst nichts von Hand neu zeichnen.

Jetzt eine **Action** – das Laden der Karten aus Supabase:

Datei: `flashCardStore.js` (**Action:** `laden`)

```
async function ladeFlashcards() {
  isLoading.value = true
  error.value = null

  const { data, error: sbError } = await supabase
    .from('flashcards')           // aus der Tabelle 'flashcards'
    .select('*')                  // alle Spalten
    .order('created_at', { ascending: false }) // neueste zuerst

  if (sbError) {
    error.value = sbError.message
  } else {
    flashcards.value = data        // Liste füllen -> Bildschirm aktualisiert sich
  }
  isLoading.value = false
}
```

In Worten: Wir sagen „Laden laeuft“, fragen Supabase nach allen Karten, und schreiben das Ergebnis in `flashcards.value`. Wegen der Reaktivitaet erscheinen die Karten danach sofort auf der Seite.

Was bedeutet `async` / `await`?

Mit der Datenbank zu sprechen dauert einen Moment. `await` heisst „warte, bis die Antwort da ist“ – ohne dass die App einfriert. Eine Funktion mit `await` darin muss mit `async` markiert sein.

CRUD ist die Abkuerzung fuer die vier Grund-Operationen einer Datenbank: **C**reate (anlegen), **R**ead (lesen), **U**ppdate (aendern), **D**eleate (loeschen).

Und so wird eine neue Karte angelegt (das „C“ und „U“ aus CRUD):

Datei: `flashCardStore.js` (Action: `hinzufuegen`)

```
async function flashcardHinzufuegen(frage, antwort, kategorie) {
  const { data, error: sbError } = await supabase
    .from('flashcards')
    .insert([ { frage, antwort, kategorie } ]) // neue Zeile in der DB
    .select()

  if (sbError) {
    error.value = sbError.message
  } else {
    flashcards.value.unshift(data[0]) // vorne in die Liste einfuegen
  }
}
```

`unshift` setzt die neue Karte an den Anfang der Liste – so erscheint sie sofort oben, ohne alles neu von der Datenbank zu laden.

Am Ende gibt der Store alles zurueck, was die Komponenten benutzen duerfen. Der Zusatz `{ persist: true }` ist das Persistenz-Plugin aus Kapitel 2:

Datei: `flashCardStore.js` (Rueckgabe)

```
return {
  flashcards, isLoading, error,
  ladeFlashcards, flashcardHinzufuegen,
  flashcardLoeschen, gelerntMarkieren
}, { persist: true } // Daten ueberleben den Seiten-Neustart
```

4.2 Das gleiche Muster ueberall

Sobald du diesen einen Store verstanden hast, kennst du alle. Deine drei Stores sind nach demselben Bauplan gebaut – nur fuer unterschiedliche Daten:

Store	Verwaltet	Besonderheit
flashCardStore	Die Lernkarten	Action <code>gelerntMarkieren</code> setzt das Feld 'gelernt' (Update).
notesStore	Die Notizen	Hat zusaetzlich <code>notizAktualisieren</code> ; speichert eine Ordner-Farbe.
questionStore	Die Fragen	Hat Getters: <code>'tagesfrage'</code> und <code>'nochNichtAngezeigt'</code> (berechnete Werte).

Ein kurzer Blick auf die **Getters** im `questionStore` zeigt, was berechnete Werte sind:

Datei: src/stores/questionStore.js (Getters)

```
const tagesfrage = computed(() => {
  if (questions.value.length === 0) return null
  return questions.value[tagesfrageIndex.value]
})

const nochNichtAngezeigt = computed(() => {
  return questions.value.filter(q => !q.angezeigt_am)
})
```

computed erstellt einen Wert, der sich automatisch neu berechnet, sobald sich die zugrunde liegenden Daten ändern. nochNichtAngezeigt ist also immer die aktuelle Liste der Fragen ohne Anzeige-Datum – ohne dass du sie selbst pflegen musst.

4.3 So fließen die Daten durch das Haus

Damit alles zusammenpasst, hier der typische Weg der Daten – von einer Komponente bis in die Datenbank und wieder zurück auf den Bildschirm:

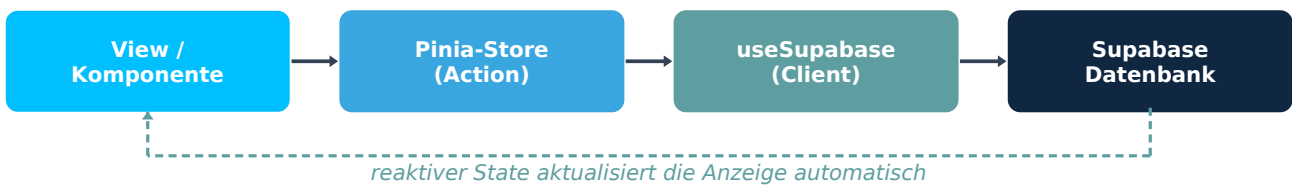


Abbildung 3: Der Datenfluss. Die Komponente ruft eine Action im Store auf, der Store spricht mit Supabase, und das reaktive Ergebnis erscheint von selbst auf der Seite.

KAPITEL 5

Der Innenausbau - das SCSS-Design-System

Jetzt kommt die Einrichtung: das Aussehen. Statt jede Farbe und jeden Abstand ueberall einzeln festzulegen, hast du ein **Design-System** gebaut. Das ist eine zentrale Sammlung von Bauvorschriften – einmal definiert, ueberall genutzt. So sieht die ganze App einheitlich aus.

Was ist SCSS?

SCSS ist eine erweiterte Form von CSS. Es kann Dinge, die normales CSS nicht kann: **Variablen** (Werte mit Namen), Verschachtelung und das Aufteilen in mehrere Dateien.

Eine Datei, die mit einem Unterstrich beginnt (z. B. `_variables.scss`), ist ein **Partial** – ein Teilstueck, das in eine Hauptdatei eingebunden wird.

5.1 Die Schaltzentrale: main.scss

Die `main.scss` ist der Verteiler. Mit `@use` zieht sie alle Teilstuecke zusammen und setzt danach ein paar globale Grund-Regeln.

Datei: `src/styles/main.scss` (Auszug)

```
@use './variables' as *;
@use './dark-light';
@use './glassmorphism';
@use './typography';

*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'AmyraFont', sans-serif;
  background-color: var(--color-bg);
  color: var(--color-text);
}
```

5.2 Die Material-Liste: `_variables.scss`

Hier liegen alle Bau-Werte als **SCSS-Variablen** (erkennbar am `$`): Farben, Abstaende, Rundungen, Schriftgroessen. Ein kleiner Ausschnitt:

Datei: src/styles/_variables.scss (Auszug)

```
// --- Glassmorphismus-Farben ---
$color-deep-sky-blue: #00BFFF;
$color-light-sky-blue: #87CEFA;

// --- Abstaende ---
$spacing-xs: 0.25rem;
$spacing-sm: 0.5rem;
$spacing-md: 1rem;
$spacing-lg: 1.5rem;

// --- Glassmorphismus-Werte ---
$glass-blur: blur(16px);
$radius-md: 16px;
```

Der Vorteil: Willst du z. B. die Hauptfarbe aendern, aenderst du sie an genau *einer* Stelle – und das ganze Haus zieht mit.

5.3 Hell und Dunkel: _dark-light.scss

Hier werden die SCSS-Variablen in **CSS-Variablen** umgewandelt (erkennbar am --). Der Trick: Es gibt zwei Saetze – einen fuer hell (:root) und einen fuer dunkel (:root.dark).

Datei: src/styles/_dark-light.scss (Auszug)

```
// --- Light Mode ---
:root {
  --color-bg:    #{$bg-light};
  --color-text:  #{$text-light};
  --color-primary: #{$color-deep-sky-blue};
  --glass-bg:    #{$glass-bg-light};
}

// --- Dark Mode ---
:root.dark {
  --color-bg:    #{$bg-dark};
  --color-text:  #{$text-dark};
  --color-primary: #{$color-deep-sky-blue};
  --glass-bg:    #{$glass-bg-dark};
}
```

Schaltet jemand auf Dark-Mode um (Kapitel 10), bekommt das html-Element die Klasse dark – und ab da gelten automatisch die dunklen Werte. Weil ueberall var(--color-bg) statt fester Farben steht, faerbt sich die ganze App auf einen Schlag um.

SCSS-Variable vs. CSS-Variable – der Unterschied

SCSS-Variablen (\$farbe) existieren nur beim Bauen. Danach sind sie „eingebacken“ und nicht mehr aenderbar.

CSS-Variablen (--farbe) leben im Browser und koennen sich *waehrend* der Nutzung aendern – perfekt fuer den Wechsel zwischen Hell und Dunkel.

5.4 Die Glas-Optik: _glassmorphism.scss

Das auffaelligste Design-Merkmal deiner App ist der **Glassmorphismus** – milchige, leicht durchsichtige Flaechen wie Milchglas. Das steckt in wiederverwendbaren Klassen wie .glass und .neu-button.

Datei: src/styles/_glassmorphism.scss (Auszug)

```
.glass {
  background: var(--glass-bg);
  backdrop-filter: blur(16px); // macht den Hintergrund milchig
  -webkit-backdrop-filter: blur(16px);
  border: $glass-border;
  box-shadow: var(--glass-shadow);
  border-radius: $radius-md;
}

.neu-button {
  background: var(--neu-bg);
  box-shadow: var(--neu-shadow); // weicher 3D-Schatten (Neumorphismus)
  border: none;
  &:active { transform: scale(0.97); } // beim Klick leicht schrumpfen
}
```

Das Herz ist backdrop-filter: blur(): Es macht alles, was *hinter* der Fläche liegt, unscharf – der typische Glas-Effekt. Das &:active ist SCSS-Verschachtelung und bedeutet „wenn dieser Button gerade gedrückt wird“.

KAPITEL 6

Die Moebel - wiederverwendbare Komponenten

Komponenten sind die Moebelstuecke deiner App: einmal gebaut, ueberall einsetzbar. Eine Komponente ist eine `.vue`-Datei mit drei Teilen - `<script>` (die Logik), `<template>` (das Aussehen/HTML) und `<style>` (das CSS, das nur fuer diese Komponente gilt).

Props und Emits - wie Komponenten miteinander reden

Props (von „properties“) sind Daten, die eine *Eltern*-Komponente nach *unten* an ein Kind weitergibt. Beispiel: die Liste reicht eine einzelne Karte an die Karten-Komponente.

Emits sind Nachrichten, die ein *Kind* nach *oben* sendet. Beispiel: „Ich wurde geloesch!“ Das Kind aendert die Daten nicht selbst - es meldet nur den Wunsch.

6.1 Die Flashcard - eine Karte zum Umdrehen

FlashCard.vue zeigt eine einzelne Karte mit 3D-Dreh-Effekt. Die Logik ist klein: Sie bekommt eine Karte als **Prop**, merkt sich, ob sie umgedreht ist, und sendet per **Emit** Wuensche nach oben.

Datei: `src/components/flashcard/FlashCard.vue` (Script)

```
const props = defineProps({
  flashcard: { type: Object, required: true } // Karte kommt von oben
})
const emit = defineEmits(['gelernt-toggle', 'loeschen']) // Wuensche nach oben

const istUmgedreht = ref(false)
function umdrehen() { istUmgedreht.value = !istUmgedreht.value }

function gelerntToggle() {
  emit('gelernt-toggle', props.flashcard.id, !props.flashcard.gelernt)
}
function loeschen() { emit('loeschen', props.flashcard.id) }
```

Der Dreh-Effekt selbst ist reines CSS: Die Karte wird um 180 Grad gedreht, sobald die Klasse `card--flipped` aktiv ist.

Datei: `FlashCard.vue` (Style, gekuerzt)

```
.card { transform-style: preserve-3d; transition: transform 0.6s ease; }
.card--flipped { transform: rotateY(180deg); } // dreht die Karte
.card__front, .card__back { backface-visibility: hidden; } // Rueckseite verstecken
```

6.2 Das Formular und die Liste - ein eingespieltes Team

FlashCardForm.vue sammelt Eingaben (Frage, Antwort, Kategorie) und sendet sie per Emit nach oben - es speichert nichts selbst. So bleibt es ein sauberes, wiederverwendbares Moebelstueck.

Datei: `src/components/flashcard/FlashCardForm.vue` (Script)

```
const emit = defineEmits(['hinzufuegen'])
const frage = ref(''); const antwort = ref(''); const kategorie = ref('')

function absenden() {
  if (!frage.value.trim() || !antwort.value.trim()) return
  emit('hinzufuegen', {
    frage: frage.value.trim(),
    antwort: antwort.value.trim(),
    kategorie: kategorie.value.trim()
  })
  frage.value = ''; antwort.value = ''; kategorie.value = '' // Felder leeren
}
```

FlashCardList.vue wiederum nimmt die ganze Liste als Prop und zeichnet fuer jede Karte eine FlashCard. Das geschieht mit v-for – einer Schleife im Template.

Datei: `src/components/flashcard/FlashCardList.vue` (Template)

```
<FlashCard
  v-for="flashcard in flashcards"
  :key="flashcard.id"
  :flashcard="flashcard"
  @gelernt-toggle="gelerntToggle"
  @loeschen="loeschen"
/>
```

Drei wichtige Vue-Kuerzel im Template

v-for = wiederhole dieses Element fuer jeden Eintrag einer Liste.

:key = gib jedem Eintrag eine eindeutige Nummer, damit Vue sie auseinanderhalten kann.

:flashcard (mit Doppelpunkt) gibt eine Prop weiter; @loeschen (mit @) hoert auf einen Emit.

6.3 Der Schreibtisch: NoteEditor.vue mit TipTap

Damit man Notizen huebsch formatieren kann (fett, Farben, Listen, Textmarker), nutzt du den **Rich-Text-Editor TipTap**. „Rich Text“ heisst „angereicherter Text“ – also Text mit Formatierung statt nur reinem Text. Der Editor wird mit Erweiterungen zusammengesteckt.

Datei: `src/components/notes/NoteEditor.vue` (Auszug)

```
const editor = useEditor({
  content: props.modelValue,
  extensions: [
    StarterKit, Underline, TextStyle,
    Color, Highlight.configure({ multicolor: true })
  ],
  onUpdate: ({ editor }) => {
    emit('update:modelValue', editor.getHTML()) // gibt formatiertes HTML zurueck
  }
})
```

Jeder Werkzeug-Knopf ruft einen TipTap-Befehl auf, z. B. fett: `editor.chain().focus().toggleBold().run()`. Beim Tippen sendet `onUpdate` den Inhalt als HTML nach oben – ueber `update:modelValue`, das ist das Vue-Muster fuer v-model (Zwei-Wege-Bindung).

6.4 Die UI-Komponenten

Im Ordner `components/ui/` liegen die Rahmen-Bauteile. `AppNavbar.vue` ist die Navigationsleiste mit allen Links und dem Hell/Dunkel-Schalter (Kapitel 10). `AppHeader.vue` und `AppFooter.vue` sind aktuell bewusst leere Platzhalter – Rohbau-Rahmen, die später gefüllt werden können. `NoteCard.vue` zeigt eine einzelne Notiz als Kachel mit farbigem Ordner-Streifen.

KAPITEL 7

Die Raeume - deine Views im Steckbrief

Views sind die Raeume des Hauses: ganze Seiten, die der Router (Kapitel 10) ueber Tueren erreichbar macht. Jede View setzt die Moebel (Komponenten) und Leitungen (Stores) zusammen. Hier jede deiner neun Views als kompakter Steckbrief.

HomeView - die Eingangshalle

Route: /

Zweck: Begrueessung plus Ueberblick: zeigt Zaehler fuer Karten, Notizen, Fragen und Gelerntes sowie Schnellzugriff-Kacheln.

Nutzt: alle drei Stores; laedt beim Oeffnen alle Daten via onMounted.

Besonderer Kniff: Ein computed-Wert zaehlt die gelernten Karten: filtert die Liste und nimmt die Laenge.

LibraryView - die Bibliothek

Route: /library

Zweck: Das Herzstueck: Karten anlegen (Formular), durchsuchen, nach Kategorie filtern, lernen/loeschen.

Nutzt: flashCardStore, FlashCardForm, FlashCardList.

Besonderer Kniff: Zwei computed-Werte: 'kategorien' sammelt alle vorkommenden Kategorien (mit new Set ohne Doppelte), 'gefilterteFlashcards' kombiniert Suche und Kategorie-Filter.

LearnView - das Lernzimmer

Route: /learn

Zweck: Lern-Modus: Karten nacheinander durchgehen, umdrehen, 'gewusst/nicht gewusst' antippen, am Ende eine Auswertung.

Nutzt: flashCardStore, useRouter (leitet zur Bibliothek um, wenn keine Karten da sind).

Besonderer Kniff: Ein Fortschrittsbalken via computed; ein watch setzt die Karte bei jedem Wechsel zurueck auf die Vorderseite.

NotesView - das Arbeitszimmer

Route: /notes

Zweck: Notizen erstellen und bearbeiten mit dem TipTap-Editor; farbiges Ordner-System mit Filter.

Nutzt: notesStore, NoteEditor, NoteCard.

Besonderer Kniff: Eine Ordner-Palette mit 8 Farben; ein 'bearbeitungsModus' entscheidet, ob gespeichert oder aktualisiert wird.

QuestionView - die Tagesfrage

Route: </questions>

Zweck: Zeigt taeglich eine Lernfrage als Flip-Karte; ist die Datenbank leer, erzeugt die KI automatisch eine Frage.

Nutzt: questionStore, useToast, der /api/chat-Endpunkt.

Besonderer Kniff: Ein Tageslimit (max. 20) wird ueber localStorage und das heutige Datum gezaehlt und bei einem neuen Tag zurueckgesetzt.

FragenkatalogView - das Pruefungszimmer

Route: </katalog>

Zweck: Zufaellige Pruefungsfragen von der KI; man tippt die Antwort in den TipTap-Editor; ein Coach bewertet sie.

Nutzt: questionStore, useAnswerValidation, NoteEditor, useToast.

Besonderer Kniff: 15 Themen werden zufaellig gemischt; das 3-Versuche-System (Kapitel 9) entscheidet ueber richtig, falsch oder Antwort aufdecken.

FragenBibliothekView - das Archiv

Route: </fragen-bibliothek>

Zweck: Listet alle gespeicherten Fragen; Antworten lassen sich aufklappen, durchsuchen und loeschen.

Nutzt: questionStore, useToast.

Besonderer Kniff: Ein 'aufgeklappteld' merkt sich, welche eine Antwort gerade offen ist (auf-/zuklappen).

SearchView - die Suchstation

Route: </search>

Zweck: Globale Suche ueber Karten, Notizen UND Fragen gleichzeitig.

Nutzt: alle drei Stores, useDebounceFn (VueUse), useRouter.

Besonderer Kniff: Debounce mit 300 ms: erst kurz nach dem letzten Tastendruck wird gesucht - das spart Rechenarbeit.

StatisticsView - das Kontrollzimmer

Route: </statistics>

Zweck: Zeigt Quoten und Fortschrittsbalken: wie viele Karten gelernt, wie viele Fragen beantwortet sind.

Nutzt: alle drei Stores.

Besonderer Kniff: Mehrere computed-Werte berechnen Prozent-Quoten und fangen dabei die Division durch Null ab.

KAPITEL 8

Sechs Views im Detail - Schritt fuer Schritt

In Kapitel 7 hast du alle Raeume von aussen gesehen. Jetzt gehen wir in sechs davon hinein und schauen uns den echten Code an. Du wirst merken: Es sind immer dieselben Bausteine aus den Kapiteln 4 bis 6 - Stores, computed-Werte, Komponenten - nur jedes Mal anders zusammengesetzt. Wer eine View versteht, versteht alle.

Der gemeinsame Bauplan jeder View

1. **Importe** oben: welche Stores und Komponenten brauche ich?
2. **onMounted**: beim Oeffnen der Seite die Daten aus Supabase laden.
3. **State & computed**: lokale Eingaben (z. B. Suchbegriff) und daraus berechnete Listen.
4. **Funktionen**: was bei Klicks passiert - meist ein Aufruf einer Store-Action.
5. **Template**: das Ganze sichtbar machen, oft mit v-if (anzeigen/verstecken) und v-for (Liste).

8.1 HomeView - das Schaufenster

Die HomeView holt sich alle drei Stores und laedt beim Oeffnen deren Daten. So kann sie sofort Zaehler anzeigen. onMounted ist ein **Lifecycle-Hook** - eine Funktion, die automatisch laeuft, sobald die Seite aufgebaut ist.

Datei: src/views/HomeView.vue (Script)

```
const flashCardStore = useFlashcardStore()
const notesStore = useNotesStore()
const questionStore = useQuestionStore()

onMounted(async () => {
  await flashCardStore.ladeFlashcards() // Karten holen
  await notesStore.ladeNotes()         // Notizen holen
  await questionStore.ladeFragen()      // Fragen holen
})

const gelerntAnzahl = computed(() =>
  flashCardStore.flashcards.filter(f => f.gelernt).length
)
```

Der computed-Wert gelerntAnzahl filtert alle Karten heraus, bei denen gelernt wahr ist, und zaehlt sie. Im Template werden die Zahlen direkt aus den Stores gelesen - mit doppelten geschweiften Klammern, dem „Mustache“-Format:

Datei: HomeView.vue (Template-Auszug)

```
<span class="home__statistik-zahl">{{ flashCardStore.flashcards.length }}</span>
<span class="home__statistik-label">Flashcards</span>
...
<span class="home__statistik-zahl">{{ gelerntAnzahl }}</span>
<span class="home__statistik-label">Gelernt</span>
```

Was sind die doppelten geschweiften Klammern {{ }}?

Das ist **Interpolation**. Alles zwischen {{ }} wird von Vue berechnet und als Text eingesetzt. Ändert sich der Wert, aktualisiert Vue den Text automatisch.

8.2 LibraryView - anlegen, suchen, filtern

Die Bibliothek ist die anspruchsvollste der drei Listen-Views. Sie kombiniert zwei computed-Werte: eine Liste der vorhandenen Kategorien und eine gefilterte Kartenliste.

Datei: src/views/LibraryView.vue (computed)

```
const kategorien = computed(() => {
  const alle = flashcardStore.flashcards
    .map(f => f.kategorie) // nur die Kategorie jeder Karte
    .filter(k => k && k.trim() !== '') // leere wegwerfen
  return [...new Set(alle)] // Doppelte entfernen
})

const gefilterteFlashcards = computed(() => {
  return flashcardStore.flashcards.filter(f => {
    const suchTreffer =
      f.frage.toLowerCase().includes(suchbegriff.value.toLowerCase()) ||
      f.antwort.toLowerCase().includes(suchbegriff.value.toLowerCase())
    const kategorieTreffer =
      aktiveKategorie.value === '' || f.kategorie === aktiveKategorie.value
    return suchTreffer && kategorieTreffer // beides muss passen
  })
})
```

Der Trick mit new Set: Ein **Set** ist eine Liste, die jeden Wert nur einmal enthält. So bekommst du jede Kategorie genau einmal als Filter-Knopf. gefilterteFlashcards zeigt nur Karten, die *gleichzeitig* zum Suchtext und zur gewählten Kategorie passen.

Die View speichert selbst nichts – sie reicht jeden Wunsch an den Store weiter. Das hast du in Kapitel 6 als Muster „Kind meldet nach oben“ gesehen:

Datei: LibraryView.vue (Handler + Template)

```
async function flashcardHinzufuegen(daten) {
  await flashcardStore.flashcardHinzufuegen(daten.frage, daten.antwort, daten.kategorie)
}

<!-- Template: Formular meldet 'hinzufuegen', Liste bekommt die gefilterten Karten -->
<FlashCardForm @hinzufuegen="flashcardHinzufuegen" />
...
<p v-if="flashcardStore.isLoading">Laedt...</p>
<FlashCardList v-else :flashcards="gefilterteFlashcards"
  @gelernt-toggle="gelerntToggle" @loeschen="loeschen" />
```

Das v-if="isLoading" / v-else-Paar zeigt während des Ladens „Laedt...“ und danach die Liste. So sieht man nie eine leere Seite, obwohl die Daten noch unterwegs sind.

8.3 LearnView - der Lern-Durchlauf

Die LearnView führt dich Karte für Karte durch den Stapel. Sie merkt sich, an welcher Stelle du bist, und zählt richtig und falsch mit.

Datei: src/views/LearnView.vue (State + Schutz)

```
const aktuellerIndex = ref(0) // welche Karte gerade dran ist
const richtig = ref(0)
const falsch = ref(0)
const fertig = ref(false)

onMounted(async () => {
  await flashcardStore.ladeFlashcards()
  if (flashcardStore.flashcards.length === 0) {
    router.push('/library') // keine Karten? -> zur Bibliothek schicken
  }
})

const fortschritt = computed(() =>
  Math.round((aktuellerIndex.value / flashcardStore.flashcards.length) * 100)
)
```

Praktisch: Gibt es gar keine Karten, schickt `router.push('/library')` dich automatisch zum Anlegen. Der `fortschritt` wird in Prozent berechnet und steuert die Breite des Balkens im Template (`:style="{ width: fortschritt + '%' }"`).

Beim Weiterblättern passiert die ganze Lern-Logik. Und ein **watch** sorgt dafür, dass jede neue Karte wieder mit der Vorderseite startet:

Datei: LearnView.vue (Logik)

```
watch(aktuellerIndex, () => {
  istUmgedreht.value = false // bei neuer Karte: nicht umgedreht
})

function naechsteKarte(gewusst) {
  if (gewusst) {
    richtig.value++
    flashcardStore.gelerntMarkieren(aktuelleKarte.value.id, true) // in DB merken
  } else {
    falsch.value++
  }
  if (aktuellerIndex.value < flashcardStore.flashcards.length - 1) {
    aktuellerIndex.value++ // naechste Karte
  } else {
    fertig.value = true // letzte Karte -> Auswertung zeigen
  }
}
```

Was ist ein „watch“?

Ein `watch` („Beobachter“) passt auf eine Variable auf und führt Code aus, sobald sie sich ändert. Hier: Sobald `aktuellerIndex` wechselt, wird die Karte zurück auf die Frage-Seite gedreht.

8.4 NotesView - Notizen mit Ordner-System

Die `NotesView` ist die einzige View, die *denselben* Bereich zum Anlegen **und** Bearbeiten nutzt. Ob gespeichert oder aktualisiert wird, entscheidet ein Schalter namens `bearbeitungsModus`.

Datei: src/views/NotesView.vue (Ordner-Palette)

```
const ORDNER_PALETTE = [
  { schluessel: 'cyan',    label: 'Cyan',    farbe: 'var(--ordner-cyan)' },
  { schluessel: 'meerblau', label: 'Meerblau', farbe: 'var(--ordner-meerblau)' },
  // ... insgesamt 8 Farben ...
]

const gefilterteNotes = computed(() => {
  if (!filterOrdner.value) return notesStore.notes // kein Filter -> alle
  return notesStore.notes.filter(n => n.farbe === filterOrdner.value)
})
```

Jeder Ordner ist nur ein Schluessel (z. B. 'cyan') plus eine CSS-Variable als Farbe – genau die aus deinem Design-System in Kapitel 5. Jetzt der Kern: die eine Speichern-Funktion, die zwei Wege kennt.

Datei: NotesView.vue (speichern: anlegen ODER aktualisieren)

```
async function speichern() {
  if (!titel.value.trim()) return

  if (bearbeitungsModus.value && aktiveNotizId.value) {
    // Wir bearbeiten eine bestehende Notiz -> UPDATE
    await notesStore.notizAktualisieren(
      aktiveNotizId.value, titel.value.trim(), inhalt.value, gewaehlterOrdner.value
    )
  } else {
    // Neue Notiz -> INSERT
    await notesStore.notizHinzufuegen(
      titel.value.trim(), inhalt.value, gewaehlterOrdner.value
    )
  }
  neueNotiz() // Formular leeren
}
```

Beim Klick auf eine Notiz füllt `notizBearbeiten` das Formular mit den alten Werten, schaltet den Bearbeitungsmodus an und scrollt sanft nach oben zum Formular (`scrollIntoView`). Der Inhalt wird ueber `<NoteEditor v-model="inhalt" />` mit dem TipTap-Editor aus Kapitel 6 verbunden.

8.5 QuestionView - die Tagesfrage mit Tageslimit

Diese View hat zwei Besonderheiten, die du sonst nirgends findest: Sie kann sich per KI selbst eine Frage erzeugen, und sie begrenzt dich auf 20 Fragen pro Tag. Das Tageslimit merkt sie sich im `localStorage` – einem kleinen Speicher direkt im Browser.

Datei: src/views/QuestionView.vue (Tageslimit)

```
const HEUTE_KEY = 'tagesfrage_datum'
const ZAEHLER_KEY = 'tagesfrage_zaebler'
const MAX_FRAGEN = 20

function pruefeHeutigenStatus() {
  const heute = new Date().toISOString().split('T')[0] // z.B. '2026-06-21'
  const gespeichertesDatum = localStorage.getItem(HEUTE_KEY)

  if (gespeichertesDatum === heute) {
    tagesZaebler.value = parseInt(localStorage.getItem(ZAEHLER_KEY) ?? '0')
  } else {
    // neuer Tag -> Zaebler zurueck auf 0
    tagesZaebler.value = 0
    localStorage.setItem(HEUTE_KEY, heute)
    localStorage.setItem(ZAEHLER_KEY, '0')
  }
  limitUeberschritten.value = tagesZaebler.value >= MAX_FRAGEN
}
```

Der Trick: Es wird das heutige Datum als Text gespeichert. Ist beim naechsten Besuch ein neuer Tag, wird der Zaebler zurueckgesetzt – so bekommst du jeden Tag wieder frische 20 Fragen.

Ist die Datenbank leer, holt sich die View per KI selbst eine Frage. Sie bittet die KI um reines JSON und wandelt die Antwort mit `JSON.parse` in ein Objekt um:

Datei: QuestionView.vue (Auto-Generierung, gekuerzt)

```
const response = await fetch('/api/chat', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    messages: [{ role: 'user', content:
      'Erstelle EINE Lernfrage ueber Webentwicklung ... Antworte NUR als JSON: {frage, antwort}' }]
  })
})
const data = await response.json()
const parsed = JSON.parse(data.content?.[0]?.text ?? '') // Text -> Objekt
await questionStore.frageHinzufuegen(parsed.frage, parsed.antwort, 'Webentwicklung')
```

Beim Klick auf „Verstanden“ markiert `beantwortet()` die Frage in Supabase als gesehen, erhoeht den Tageszaehler und zeigt einen passenden Toast – bei Frage 20 eine Stopp-Meldung.

localStorage vs. Datenbank - wann was?

Supabase speichert die Inhalte dauerhaft und geraeteuebergreifend (deine Fragen selbst).

localStorage speichert kleine, geraet-lokale Notizen wie „heute schon 5 Fragen gemacht“ oder die DSGVO-Zustimmung. Es bleibt nur in genau diesem Browser.

8.6 FragenBibliothekView – das durchsuchbare Archiv

Hier liegen alle gespeicherten Fragen. Die View kann suchen, einzelne Antworten auf- und zuklappen und Fragen loeschen. Das Suchen laeuft wieder ueber einen computed-Wert.

Datei: src/views/FragenBibliothekView.vue (Suche + Aufklappen)

```
const gefilterteFragen = computed(() => {
  if (!suchbegriff.value.trim()) return questionStore.questions
  const begriff = suchbegriff.value.toLowerCase()
  return questionStore.questions.filter(q =>
    q.frage?.toLowerCase().includes(begriff) ||
    q.antwort?.toLowerCase().includes(begriff) ||
    q.kategorie?.toLowerCase().includes(begriff)
  )
})

function toggleAntwort(id) {
  // gleiche Frage nochmal -> zuklappen, sonst diese aufklappen
  aufgeklappteId.value = aufgeklappteId.value === id ? null : id
}
```

Das Fragezeichen in `q.frage?.toLowerCase()` ist der **Optional-Chaining-Operator**: Er verhindert einen Absturz, falls ein Feld einmal leer ist. `toggleAntwort` merkt sich in `aufgeklappteId`, welche *eine* Antwort gerade offen ist – klickt man dieselbe erneut, klappt sie zu.

Beim Loeschen wird die Store-Action aufgerufen und ein Toast gezeigt; die kleine Hilfsfunktion `formatDatum` macht aus dem technischen Datum ein lesbares deutsches:

Datei: FragenBibliothekView.vue (loeschen + Datum)

```
async function loeschen(id) {
  await questionStore.frageLoeschen(id)
  toast.success('Frage geloescht.')
  if (aufgeklappteId.value === id) aufgeklappteId.value = null
}

function formatDatum(isoString) {
  if (!isoString) return null
  return new Date(isoString).toLocaleDateString('de-DE') // -> 21.6.2026
}
```

Schoenes Detail: die Slide-Animation

Das Auf- und Zuklappen der Antwort ist in `<Transition>` gewickelt. Vue blendet den Bereich dadurch sanft ein und aus, statt ihn hart erscheinen zu lassen.

KAPITEL 9

Der smarte Helfer - BlueBooster & KI-Validierung

Das Besondere an Flashback ist die KI-Unterstützung. Sie taucht an zwei Stellen auf: als Chat (**BlueBooster**) und als **Antwort-Prüfer** im Fragenkatalog. Beide reden über denselben Weg mit der KI: `fetch('/api/chat')` -> Proxy -> Express -> Anthropic (Abbildung 2 aus Kapitel 3).

8.1 BlueBooster - der Chat-Assistent

Die Logik steckt im Composable `useBlueBooster.js`. Es hält die Nachrichten, einen **System-Prompt** (die „Charakter-Beschreibung“ der KI) und die Funktion zum Senden. Ein **Prompt** ist die Anweisung, die man der KI gibt.

Datei: `src/composables/useBlueBooster.js` (Senden)

```
async function nachrichtSenden(text) {
  if (!text.trim() || laedt.value) return
  nachrichten.value.push({ role: 'user', content: text.trim() })
  laedt.value = true

  try {
    const response = await fetch('/api/chat', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        system: SYSTEM_PROMPT,
        messages: nachrichten.value.map(n => ({ role: n.role, content: n.content }))
      })
    })
    const data = await response.json()
    const antwort = data.content?.[0]?.text ?? 'Entschuldigung, ...'
    nachrichten.value.push({ role: 'assistant', content: antwort })
  } catch (err) {
    fehler.value = 'Verbindungsfehler. Bitte versuche es erneut.'
  } finally {
    laedt.value = false
  }
}
```

Die Anzeige übernimmt `BlueBooster.vue`: ein Smiley-Knopf unten rechts, ein Chat-Fenster, ein **DSGVO-Hinweis** vor dem ersten Nutzen und Markdown-Darstellung der Antworten (über das Paket `marked`).

Datenschutz ist eingebaut (DSGVO)

BlueBooster speichert keinen Chatverlauf dauerhaft – er lebt nur im Arbeitsspeicher und ist nach dem Neuladen weg.

Das Einzige, was gemerkt wird, ist die einmalige Einwilligung in `localStorage` unter dem Schlüssel `blueBooster_dsgvo_akzeptiert`.

8.2 Das 3-Versuche-System: `useAnswerValidation.js`

Im Fragenkatalog prüft die KI, ob deine Antwort inhaltlich stimmt. Das Composable `useAnswerValidation.js` schickt Frage, Musterantwort und deine Antwort an die KI und bittet um genau ein Wort: RICHTIG oder FALSCH.

Datei: `src/composables/useAnswerValidation.js` (Kern)

```
const MAX_ATTEMPTS = 3

async function validateAnswer(frageId, frageText, musterantwort, nutzerantwort) {
  if (attempts.value[frageId] >= MAX_ATTEMPTS) return
  isValidating.value = true
  // ... fetch('/api/chat') mit Bewertungs-Prompt ...
  const urteil = data.content?.[0]?.text?.trim().toUpperCase()

  if (urteil === 'RICHTIG') {
    validationStatus.value[frageId] = 'correct'
  } else {
    attempts.value[frageId] += 1
    if (attempts.value[frageId] >= MAX_ATTEMPTS) {
      validationStatus.value[frageId] = 'revealed' // Antwort aufdecken
    } else {
      validationStatus.value[frageId] = 'wrong'
    }
  }
}
```

In Worten: Ist die Antwort richtig, wird ein blauer Rahmen gesetzt. Ist sie falsch, zählt ein Versuchszaehler hoch. Nach drei Fehlversuchen schaltet der Status auf 'revealed' – und die Musterantwort wird gezeigt. Genau das Verhalten, das auch im System-Prompt von BlueBooster steht.

Was ist ein „Composable“?

Ein **Composable** ist eine wiederverwendbare Funktion, die Vue-Logik bundelt (Name beginnt per Konvention mit `use`). Stores kümmern sich um *geteilte Daten*, Composables um *wiederverwendbares Verhalten*. Deine vier Composables: `useSupabase`, `useBlueBooster`, `useAnswerValidation` und `useFlashcards`.

KAPITEL 10

Komfort & Technik - PWA, Toasts, VueUse, Routing

9.1 Die Tueren: Vue Router

Der **Router** ist das System aus Tueren und Fluren. Er ordnet jeder Adresse („Route“) eine View zu. So wechselt die Seite, ohne dass der Browser komplett neu laedt („Single Page Application“).

Datei: `src/router/index.js` (Auszug)

```
const routes = [
  { path: '/', name: 'Home', component: HomeView },
  { path: '/library', name: 'Library', component: LibraryView },
  { path: '/learn', name: 'Learn', component: LearnView },
  { path: '/questions', name: 'Questions', component: QuestionView },
  // ... weitere Routen ...
]
const router = createRouter({ history: createWebHistory(), routes })
```

In der `AppNavbar.vue` fuehren `<RouterLink>`-Elemente zu diesen Routen. Die aktuell aktive Route wird optisch hervorgehoben.

9.2 Hell/Dunkel und Debounce: VueUse

VueUse ist eine Sammlung fertiger Helfer. Du nutzt zwei davon. Der Dark-Mode steckt in nur drei Zeilen in der Navbar:

Datei: `src/components/ui/AppNavbar.vue` (Dark-Mode)

```
import { useDark, useToggle } from '@vueuse/core'
const isDark = useDark() // liest/merkt die Vorliebe
const toggleDark = useToggle(isDark) // schaltet um
```

`useDark` setzt automatisch die Klasse `dark` am `html`-Element – wodurch die dunklen CSS-Variablen aus Kapitel 5 greifen. Der zweite Helfer, `useDebounceFn`, wird in der Suche genutzt: Er wartet 300 ms nach dem letzten Tastendruck, bevor gesucht wird – das nennt man **Debounce** („Entprellen“).

Datei: `src/views/SearchView.vue` (Debounce)

```
const debounceSearch = useDebounceFn((wert) => {
  aktiverSuchbegriff.value = wert
}, 300) // erst 300ms nach dem letzten Tastendruck
```

9.3 Hinweis-Meldungen: Vue-Toastification

Ein **Toast** ist eine kleine Meldung, die kurz oben rechts erscheint (z. B. „Frage geloescht“). In Kapitel 2 wurde das Plugin angemeldet; in einer View nutzt man es so:

Datei: Beispiel aus einer View

```
import { useToast } from 'vue-toastification'
const toast = useToast()
toast.success('Frage geloescht.')
```

9.4 Installierbar machen: die PWA

Eine **PWA** (Progressive Web App) ist eine Website, die man wie eine echte App installieren kann - mit Icon auf dem Startbildschirm und teils offline-faehig. Das uebernimmt das vite-plugin-pwa in der vite.config.js ueber ein **Manifest** (Name, Farben, Icons) und einen **Service Worker** (der Dateien zwischenspeichert).

Datei: vite.config.js (PWA, gekuerzt)

```
VitePWA({
  registerType: 'autoUpdate',
  manifest: {
    name: 'Flashback - Deine Lernkartei',
    short_name: 'Flashback',
    theme_color: '#00BFFF',
    display: 'standalone',
    icons: [ /* 192px und 512px */ ]
  },
  workbox: {
    // API-Anfragen NIE cachen - immer live vom Server
    navigateFallbackDenylist: [/^\/api/]
  }
})
```

Clever geloest

Die KI-Anfragen (/api) werden bewusst vom Zwischenspeicher ausgenommen. Sonst koennte die App veraltete KI-Antworten ausliefern, statt frische vom Server zu holen.

KAPITEL 11

Glossar - englische Begriffe einfach erklärt

Alle wichtigen englischen Fachbegriffe aus diesem Handbuch auf einen Blick, alphabetisch sortiert.

Begriff	Bedeutung
Action	Eine Funktion in einem Store, die die Daten aendert (z. B. laden, hinzufuegen, loeschen).
async / await	„asynchron / warte“. Erlaubt zu warten, bis z. B. die Datenbank antwortet, ohne dass die App einfriert.
Backend	Der unsichtbare Hintergrund: Datenbank und Server.
Backdrop-filter	Ein CSS-Effekt, der den Hintergrund hinter einer Flaechе unscharf macht – die Glas-Optik.
Build-Tool	Ein Werkzeug, das den Code fertig zusammenbaut und ausliefert (hier: Vite).
Component (Komponente)	Ein wiederverwendbares Bauteil der Oberflaeche (eine .vue-Datei).
Composable	Eine wiederverwendbare Funktion fuer Vue-Logik; Name beginnt mit 'use'.
computed	Ein Wert, der sich automatisch neu berechnet, wenn sich seine Grunddaten aendern.
CRUD	Create, Read, Update, Delete – die vier Grund-Operationen einer Datenbank.
Debounce	„Entprellen“: kurz warten, bis der Nutzer fertig getippt hat, bevor etwas passiert.
Emit	Eine Nachricht, die ein Kind-Bauteil nach oben an sein Eltern-Bauteil sendet.
Frontend	Alles, was man im Browser sieht und bedient.
Getter	Ein berechneter Wert in einem Store (basiert auf computed).
Glassmorphism	Design-Stil mit milchig-durchsichtigen Glas-Flaechen.
import / export	Code aus einer anderen Datei hereinholen bzw. fuer andere bereitstellen.
Neumorphism	Design-Stil mit weichen 3D-Schatten, der Knoepfe „gepraegt“ wirken laesst.
Persistenz	Daten ueberleben den Seiten-Neustart (hier via pinia-plugin-persistedstate).
Plugin	Ein Zusatzbaustein, den man der App einmal bekannt macht und ueberall nutzt.
Prop	Daten, die eine Eltern-Komponente nach unten an ein Kind weitergibt.
Prompt	Die Anweisung/Frage, die man der KI gibt.
Proxy	Ein Vermittler, der Anfragen entgegennimmt und weiterleitet.
PWA	Progressive Web App – eine installierbare, teils offline-faehige Web-App.
reaktiv (ref)	Eine Variable, bei deren Aenderung die Anzeige automatisch mitzieht.
Rich-Text-Editor	Ein Editor fuer formatierten Text (fett, Farben, Listen) – hier TipTap.
Route / Router	Eine Adresse in der App / das System, das Adressen den Seiten zuordnet.
Service Worker	Ein Hintergrund-Skript, das Dateien zwischenspeichert (fuer die PWA).
State	Der aktuelle Daten-Zustand der App.
State-Management	Das geordnete Verwalten dieser Daten an zentraler Stelle (hier Pinia).
System-Prompt	Die Grund-Anweisung, die der KI ihre Rolle und Regeln vorgibt.

Begriff	Bedeutung
Tech-Stack	Die Liste aller Technologien, aus denen die App besteht.
Toast	Eine kleine, kurz eingeblendete Hinweis-Meldung.
v-for / v-model	Vue-Kuerzel: Liste durchlaufen / Zwei-Wege-Bindung von Eingaben.

Anhang - die komplette Ordnerstruktur

So ist dein Projekt aufgeräumt. Jeder Ordner hat eine klare Aufgabe – das ist der Bauplan des Hauses auf einen Blick.

```
flashcard-app/
├─ index.html           -> das leere Grundstueck (Einstiegspunkt)
├─ vite.config.js       -> Bau-Werkzeug: Vue, PWA, Proxy
├─ server.cjs           -> lokaler Express-Proxy (versteckt KI-Schluessel)
├─ package.json         -> Bauteil-Liste & Start-Kommandos
├─ .env                 -> Tresor mit Schluesseln (nie in Git!)
├─ api/
│   └─ chat.js          -> Serverless-Variante des Proxys (fuer Deployment)
└─ src/
    ├─ main.js           -> Startknopf: meldet alle Plugins an
    ├─ App.vue           -> Rohbau: Navbar + RouterView + Footer + BlueBooster
    ├─ router/
    │   └─ index.js      -> die Tueren (alle Routen)
    ├─ styles/           -> der Innenausbau (Design-System)
    │   ├─ main.scss     -> Verteiler + globale Grund-Regeln
    │   ├─ _variables.scss -> Material-Liste (Farben, Abstaende ...)
    │   ├─ _dark-light.scss -> Hell/Dunkel als CSS-Variablen
    │   ├─ _glassmorphism.scss -> Glas-Optik & Knopf-Stile
    │   └─ _typography.scss -> Schrift-Regeln
    ├─ composables/      -> wiederverwendbares Verhalten (use...)
    │   ├─ useSupabase.js -> Verbindung zur Datenbank
    │   ├─ useBlueBooster.js -> Chat-Logik
    │   ├─ useAnswerValidation.js -> 3-Versuche-Pruefung
    │   └─ useFlashcards.js -> (Platzhalter)
    ├─ stores/           -> das Leitungsnetz (Pinia)
    │   ├─ flashCardStore.js -> Karten
    │   ├─ notesStore.js   -> Notizen
    │   └─ questionStore.js -> Fragen
    ├─ components/       -> die Moebel (wiederverwendbar)
    │   ├─ flashcard/    FlashCard.vue, FlashCardForm.vue, FlashCardList.vue
    │   ├─ notes/        NoteCard.vue, NoteEditor.vue
    │   ├─ chatbot/      BlueBooster.vue
    │   └─ ui/           AppNavbar.vue, AppHeader.vue, AppFooter.vue
    └─ views/            -> die Raeume (ganze Seiten)
        ├─ HomeView.vue   └─ QuestionView.vue
        ├─ LibraryView.vue └─ FragenkatalogView.vue
        ├─ LearnView.vue  └─ FragenBibliothekView.vue
        ├─ NotesView.vue  └─ SearchView.vue
        └─ StatisticsView.vue
```

Geschafft! Dein Haus steht.

Du hast jetzt den kompletten Bauplan in der Hand – vom Fundament (Vue + Vite) ueber die Versorgung (Supabase + API-Proxy) und die Leitungen (Pinia) bis zur Einrichtung (SCSS) und den Raeumen (Views).

Mein Tipp zum Nachvollziehen: Lies das Haus von unten nach oben – starte bei `main.js`, folge einer Aktion in einen Store, von dort zu `useSupabase` und zurueck in eine View. Dann hast du den ganzen Datenfluss einmal selbst „abgelaufen“.